Embedded Linux & System Software Interview Handbook

Chapter 4 - Board Bring-up (Part 3)

Topics: BootROM, Boot Modes, SPL, U-Boot, UART Bring-up and Early Boot Debugging

4.11 Understanding the Boot Flow

Once power, clocks, reset and DDR are functional, the processor begins executing its internal BootROM. The BootROM is permanently programmed by the SoC vendor and cannot normally be modified.

Complete Boot Chain

Power ON → BootROM → SPL (Secondary Program Loader) → U-Boot → Linux Kernel → initramfs → Root File System → systemd → User Applications

4.12 BootROM

BootROM initializes the minimum hardware required to locate the boot device. Depending on SoC configuration, it may boot from eMMC, SD Card, SPI NOR Flash, NAND Flash or USB Recovery Mode.

Typical BootROM Responsibilities

• Read boot configuration pins
• Initialize internal SRAM
• Load SPL into SRAM
• Verify image signatures (Secure Boot)
• Jump to SPL

Interview Question

Q: Why doesn't BootROM directly load Linux?
A: BootROM is intentionally small. It only initializes enough hardware to load the next stage (SPL). DDR initialization and full hardware setup are performed later.

4.13 Boot Modes

Modern SoCs support multiple boot sources. Boot mode is selected using configuration straps, fuses or GPIO pins.

Common boot devices:
• eMMC (Production)
• SD Card (Development)
• SPI NOR Flash
• NAND Flash
• USB Recovery

Debug Tip

If BootROM cannot locate a valid image, many processors automatically enter USB download or recovery mode.

4.14 SPL (Secondary Program Loader)

SPL is a lightweight bootloader responsible for DDR initialization and loading the full U-Boot image into external memory.

Responsibilities of SPL

• Initialize DDR Controller
• Configure PLLs
• Initialize storage controller
• Load U-Boot into DDR
• Jump to U-Boot

4.15 U-Boot

U-Boot is the primary bootloader used in Embedded Linux systems. It provides command-line access, environment variables, boot scripting, firmware updates and kernel loading.

Common U-Boot Commands

printenv
setenv bootargs ...
saveenv
mmc list
mmc info
fatload
booti / bootm

4.16 UART Bring-up

UART is the first debugging interface engineers enable during board bring-up. It provides visibility into every boot stage long before display or networking become available.

Typical UART Output

BootROM → SPL Version → DDR Initialized → U-Boot Banner → Loading Kernel → Linux Boot Messages

If UART Shows Nothing

Possible causes:
• Wrong UART pins
• Incorrect baud rate
• Missing clock
• CPU stuck in reset
• BootROM never executed

Real Samsung Example

While bringing up a Smart TV board, engineers first validated the UART console. Once the U-Boot banner appeared consistently, they proceeded to validate DDR stability, storage initialization and kernel loading.

Debugging Boot Failures

Observe the last UART message carefully.

• No output → Power/Clock/Reset/UART issue.

• Stops before SPL → BootROM or storage issue.
• Stops in SPL → DDR initialization failure.
• Stops in U-Boot → Storage or environment issue.
• Stops during Linux → Kernel or Device Tree issue.
• Stops after kernel → Driver initialization problem.

Interview Questions

1. What is BootROM?
2. Explain the complete boot chain.
3. Why is SPL required?
4. Why is UART the first peripheral enabled?
5. How do you identify where the boot process is hanging?
6. What is the difference between BootROM and U-Boot?

Key Takeaways

Understanding the boot chain is essential for embedded Linux engineers. In interviews, explain each stage clearly and always correlate UART logs with the current execution stage. This demonstrates practical debugging experience.